

Cascaded Semantic Joins over Structured and Unstructured Data

Anna Remizova, Sihem Amer-Yahia, Silviu Maniu
CNRS, Univ. Grenoble Alpes
Grenoble, France

ABSTRACT

Answering natural-language questions that mix predicates on structured attributes (e.g. film’s release year, duration, genre) with unstructured attributes (e.g. film’s review) requires a query engine that can call a large language model (LLM) as a relational operator. Two execution strategies are commonly used: a predicate filtering strategy that pushes structured filters into SQL and evaluates the semantic predicate with one LLM call per candidate row - SUQL style approach, and a batched-join strategy that packs several rows into a single LLM call and asks the model to execute the whole predicate over a chunk of rows at once - Trummer’s approach. We show empirically that the row-wise strategy is accurate but slow while the batched strategy is faster but noticeably less accurate. We suggest utilizing a two-level cascade to introduce a fast, yet quality preserving pipeline that adds a structured SUQL-style filtering before the Trummer-style batch join and finally a cheap, pre-filter before either strategy. Applied to either rows or batches, for which the cheap stage is confident are resolved immediately, and only the ones that fall inside a Bayesian-calibrated ambiguity interval are escalated to the expensive stage. Cascading the batched join strategy with applied structured filtering produces the best, to our knowledge, result over the two tested domains: *IMDb* and *Amazon products* datasets. We provide full architecture, pseudocode, prompts, and a Beta-posterior formalization of the calibrated cascade threshold that explains this cost-quality curve.

KEYWORDS

LLM query processing, structured-unstructured query language, cascaded inference, semantic operators, cost-based optimization, table QA

1 INTRODUCTION

Consider the question

“Which movies released in 1998 have reviews expressing an overall negative, critical, or strongly unfavorable opinion of the movie?”

Answering it requires (i) a structured filter over a year column and (ii) a semantic filter over free-text review. No structured query language can express the second half, for two reasons: there is no fixed vocabulary to match against (a negative opinion can be expressed in many ways, including through negation, sarcasm, or implication) and there is no schema to index it by, since the judgment depends on the review’s overall tone rather than on any single field. It requires either a learned text classifier or, at the level of accuracy and flexibility we target here, a large language model (LLM) utilized as a query operator.

SUQL [7] addresses this class of hybrid structured/semantic question by compiling the structured half into a SQL filter and the semantic half into an `answer()` function backed by an LLM.

ELEET [13] generalizes this to a *multi-modal* operator model in which a single logical scan pulls evidence from more than one physical source and fuses it before a selection is applied. Stretto [10] reduces the cost of the semantic operator itself, via (a) Bayesian threshold calibration of the operator’s accept/reject decision and (b) a KV-cache-aware execution ladder. Trummer [12] has the LLM itself join blocks of the two collections in one call, treating this task as a semantic-structured block join problem. We formalize this as query evaluation over two independent filtering operators – a structured predicate σ_S and a semantic predicate σ_U – and build our pipeline on the two existing strategies: SUQL-style predicate pushdown and Trummer-style batched joins.

SUQL structured SQLite filter first, then one LLM `answer()` call per surviving row. High quality, but calls grow linearly with the number of rows that survive the SQLite filter, so the wall-clock time dominates when the structured selectivity isn’t small.

TRUMMER the structured relation and the semantic relation are kept as two separate collections with no structural filter. The LLM joins a block of one against a block of the other while applying both structural and semantic restrictions. Such architecture decreases dramatically the amount of the calls but is less accurate: the model must recover rows while performing a join over the entire candidate pool.

The central contribution of this paper is a two-level cascade layered on top of either baseline (CASUQL and CASTRUMMER): a cheap stage classifies each unit of work (unit is a row or a batch, depending on the implementation) and only escalates units to the expensive stage if their calculated confidence falls within an ambiguity band. Introducing such a band raises a new design question neither baseline has to answer: exactly where its two edges should sit, given that the cheap stage’s own reliability is known only from a small calibration sample. A threshold picked by hand offers no guarantee about the units it accepts or rejects on its own – set it too aggressively and the cheap stage is trusted on cases it has barely seen; set it too conservatively and nearly everything is escalated, forfeiting the savings the cascade exists to provide. We resolve this by deriving the escalation rule from a Beta posterior over the cheap stage’s empirical recall and precision, which turns each candidate threshold’s small-sample accuracy into a credible bound rather than a point estimate, and lets the band widen or narrow automatically with how much calibration evidence actually supports it (§3.8).

We evaluate all four resulting methods – the two baselines and their cascaded variants – on ten held-out questions over a ~25,000-row IMDb corpus of movies and reviews, and check whether the findings hold on a second, structurally different domain (Amazon Fashion reviews). Across both, cascading the batched-join baseline gives the best cost-quality trade-off of the four methods: on our primary IMDb benchmark it is 2.03× faster and uses 10× fewer

LLM calls than its own uncascaded baseline while simultaneously gaining +70% F1, and it is the only method that does not collapse in quality on the hardest questions in our suite. The rest of the paper details the two baselines (§3.4), the cascade architecture and its calibration (§3.5–§3.8), and the experiments supporting these findings (§4–§4.3).

2 RELATED WORK

We organize related work around the sub-problems our approach must solve.

Avoiding LLM calls on rows a cheap check could reject. SUQL [7] extends SQL with a free-text answer() function and shows that evaluating structured predicates before semantic ones (structured filtering) is the dominant lever for controlling LLM cost, since it shrinks the candidate pool before the expensive operator. Our SUQL baseline follows this structure precisely; our contribution is layering a cascade on top of the semantic operator itself, once filtering has already made the pool as small as it can.

Avoiding one LLM call per row in the first place. Even after the filtering, a per-row answer() call is one LLM invocation per surviving row. Trummer [12] instead treats semantic filtering as a block-nested-loops join: the LLM gets a batch of rows from each side at once and asked to return every match in that batch in a single call, with an adaptive batch size chosen to fit the model’s context budget. Our Trummer baseline implements this algorithm directly; CASTRUMMER keeps its batching but adds the structured filtering and calibrated escalation to the original algorithm.

Combining evidence from more than one physical source. ELEET [13] treats a semantic scan as a multi-modal operator that reads structured and unstructured evidence for the same logical entity from separate physical sources and combines it before a selection is applied. ELEET also gives a cost-aware reordering rule for a sequence of such operators, which we adopt (§3.8) to order CASTRUMMER’s three filtering stages.

Deciding when a cheap stage can be trusted. Once a cheap stage exists, a threshold is needed for when to trust it immediately versus escalate to a more expensive check. A hand-picked threshold gives no guarantee about the quality of decisions made outside it. Stretto [10] addresses this with (1) a Bayesian calibration procedure that turns a raw log-odds score into a credible interval on precision/recall via a Beta posterior fit to a calibration set, and (2) a KV-cache-aware execution ladder that reuses attention state across cascade stages. We build on (1) fully for CASUQL (CASTRUMMER currently uses a simpler variant, §3.7); (2) requires direct attention-state access incompatible with our Ollama-served backend, so we defer it.

Broader context. Predicate filtering ahead of an expensive operator traces back to classical query optimization: Selinger et al.’s cost-based join ordering for System R [11], expensive-predicate scheduling [2, 4, 5], and the per-pair join cost such filtering is meant to avoid [8] together motivate our own operator-ordering rule (§3.8). Beyond joins, text-to-SQL systems such as DAIL-SQL [3] resolve only the structured half of a question, a gap TAG [1] makes explicit; LOTUS [9] and Palimpsest [6] come closest to our setting, optimizing a whole query plan over declarative

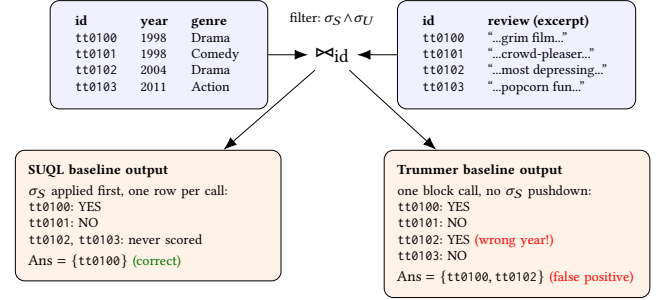


Figure 1: The running example as a join.

semantic operators with a cost-based choice of model and batching per operator.

3 APPROACH

We first formalize our query model shared by both baselines (§3.2), then describe the two baseline architectures (§3.4), the general cascade abstraction (§3.5), our two cascaded methods – CASUQL and CASTRUMMER – built on top of each baseline (§3.6–§3.7), and finally the calibration procedure that ties the two together (§3.8).

3.1 A Running Example

To illustrate the challenge, return to the question from §1: “Which movies released in 1998 have reviews expressing an overall negative opinion?” Figure 1 shows an instance of R – four rows, each merged with one review on id – built to contain one example of each row category used throughout this paper’s benchmark (§4.1): tt0100 is a *true positive*, tt0101 a *structured-semantic negative* (satisfies σ_S , not σ_U), tt0102 a *semantic-only distractor* (would satisfy σ_U , fails σ_S), and tt0103 a *double negative*. Only tt0100 belongs in $\text{Ans}(Q)$, and the figure shows what each baseline actually returns once the same merged data reaches it.

Under the SUQL baseline, σ_S is applied first, so $R_\sigma = \{\text{tt0100}, \text{tt0101}\}$ and the LLM only ever judges one row-review pair per call: a call on tt0100 returns YES, a call on tt0101 returns NO, and tt0102 and tt0103 never reach the LLM at all, since σ_S already rejected them for free. The unit of work is a single row, and σ_U is the only thing left to decide.

Under the Trummer baseline, no such filtering happens: all four rows and all four reviews are handed to the model together as one block, and the unit of work is that entire block rather than a row. A single call must now simultaneously (i) match each review back to its row by id, (ii) apply σ_S itself, since no SQL engine has done it, and (iii) apply σ_U – three jobs competing for the same forward pass instead of one. This is where the conflation failure of §3.4 comes from: a model that keeps track of the sentiment but drops the year check under this load will accept tt0102 – negative review, wrong year – exactly as readily as tt0100, and there is no separate YES/NO signal to tell that mistake apart from a correct one.

3.2 Query Model

Let R be a relation with a structured schema $\text{attr}(R)$ (a release year, genre, rating) and, for each row $r \in R$, an associated free-text field $\text{text}(r)$ (a review, a synopsis, a caption). A natural-language question Q is compiled into a pair of predicates

$$Q \mapsto (\sigma_S, \sigma_U),$$

where σ_S is a structured predicate over $\text{attrs}(R)$ expressible in relational algebra (range, equality, and set-membership conditions on numeric or categorical columns) and σ_U is a semantic predicate over $\text{text}(r)$ that cannot be parsed by any structured query language and must be delegated to an LLM. The answer to Q is

$$\text{Ans}(Q) = \{r \in R : \sigma_S(r) \wedge \sigma_U(r)\}.$$

3.3 Solution Framework

Predicate extraction ($Q \mapsto (\sigma_S, \sigma_U)$), the first box of Figure 2) is the NL-to-query step that turns Q into the structured/semantic split our framework operates on – essentially a text-to-SQL problem [3], extended to also carve out the semantic residual that no SQL dialect can express [1]. We treat this step as a black box: any component that reliably maps Q to a (σ_S, σ_U) pair would do, and improving this mapping is not the focus of this paper. In our implementation it is realized as a single one-shot LLM call constrained to output a fixed JSON schema (column, operator, value for σ_S ; a free-text restatement of the semantic condition for σ_U), and this same call is used identically across all four methods. The methods differ only in how σ_U is subsequently evaluated over the rows that σ_S admits, which is the object of the rest of this section. Two properties of this model matter for what follows:

- **Independence.** We assume σ_S and σ_U act on disjoint evidence (structured columns vs. free text), so their selectivities can be estimated separately – this is what the operator-ordering argument of §3.8 relies on.
- **Asymmetric cost.** σ_S has a constant-time SQLite predicate evaluation cost, while σ_U costs one or more LLM calls. All design decisions below aim to minimize the size of σ_U calls without decreasing its precision/recall on $\text{Ans}(Q)$.

3.4 Baseline Architectures

Given the two properties above, the central design question is how to evaluate σ_U as rarely and as cheaply as possible without losing precision or recall on $\text{Ans}(Q)$. Rather than starting from a blank slate, we ground our approach in two existing strategies that answer this question in opposite ways: SUQL’s row-wise predicate filtering, and Trummer’s batched block join, described next.

SUQL baseline Structured predicates are compiled to a WHERE clause and executed via SQLite instance, producing $R_\sigma = \{r : \sigma_S(r)\}$. For every $r \in R_\sigma$, one answer(review, question) call is issued to the LLM, which returns YES/NO. Cost is $O(|R_\sigma|)$ calls. Quality is near-maximum because every row is judged individually, in full context, with an explicitly recall-first retrieval prompt – but this is also precisely why cost scales linearly with $|R_\sigma|$ and cannot be decreased: nothing about the prompt or the call is shared across rows.

Trummer baseline The Trummer baseline applies *no* structural filtering: the structural relation and the text relation

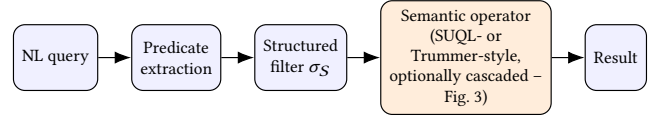


Figure 2: General schema of our approach: after structured filter, a semantic operator – SUQL- or Trummer-style, either as a baseline or wrapped in the two-level cascade of Figure 3 – resolves the semantic predicate σ_U .

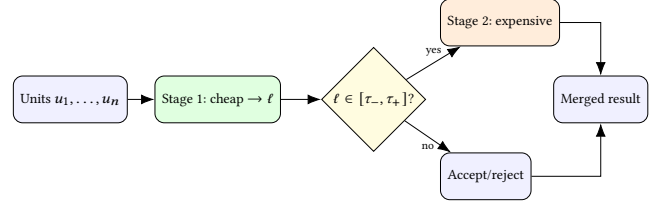


Figure 3: Two-level cascade shared by CasSuQL ($B=1$) and CasTrummer ($B=8$, batched).

are kept as two independent collections, each partitioned into blocks, and the LLM is handed one block from each side per call and asked to jointly apply $\sigma_S \wedge \sigma_U$, returning one movie_id: YES/NO line per candidate. Block size is chosen adaptively to stay under the context budget. This collapses the call count by roughly the block size, but forces the model to perform three jobs at once inside a single forward pass – (i) join the two collections on id, (ii) evaluate the structured condition it was never given a SQL engine for, and (iii) evaluate the semantic condition – which is the root cause of its recall loss (§4.2): as the candidate pool in one call grows, each row gets less of the model’s attention, so it falls back on the structured condition alone or a shallow keyword match for the semantic one.

We suggest an updated pipeline that combines the ideas from both baselines, the schema of that pipeline is shown in the Figure 2.

3.5 Cascaded Architectures

Both CasSuQL and CasTrummer add this structural idea before the semantic operator of their respective baseline: a cheap stage resolves confident units immediately, and only units whose score falls inside a calibrated ambiguity band are escalated to an expensive verification stage (Figure 3). A cascade consists of three stages:

- **Cheap** : $u \mapsto p \in (0, 1)$ is a low-cost estimator of $\Pr[\sigma_U(u) = \text{true}]$ for a unit u (a row or a batch of rows), implemented as a single call whose score is read off the token-level log-probability;
- **Scoring** maps p to log-odds $\ell = \log \frac{p}{1-p}$ and compares ℓ against a calibrated band $[\tau_-, \tau_+]$ (§3.8); units with $\ell \notin [\tau_-, \tau_+]$ are accepted or rejected on the cheap score alone;
- **Expensive** is the expensive stage – full-context judgment invoked only for units with $\ell \in [\tau_-, \tau_+]$.

3.6 CASUQL: cascaded SUQL

After the structural filtering a cheap stage gives every surviving row a single-token, logprob-based score. Rows with the score outside the calibrated band are resolved without any further LLM call; only rows inside the band reach a full answer() call. This is strictly a *subset* of the baseline's LLM calls, so CASUQL can only use fewer or equal expensive calls – but wall-clock may still grow substantially, because the cheap stage's own latency exceeds the expensive stage's latency.

Algorithm 1 CASUQL (row-level cascade, $B=1$)

Require: rows R_σ surviving σ_S ; band $[\tau_-, \tau_+]$

```

1: Ans  $\leftarrow \emptyset$ 
2: for  $r \in R_\sigma$  do
3:    $p \leftarrow \text{Cheap}(r, \sigma_U)$   $\triangleright$  single constrained-decoding call;
   read Yes/No token logprob
4:    $\ell \leftarrow \log \frac{p}{1-p}$ 
5:   if  $\ell \geq \tau_+$  then
6:     Ans  $\leftarrow$  Ans  $\cup \{r\}$   $\triangleright$  accepted on cheap score alone
7:   else if  $\ell \leq \tau_-$  then
8:     reject  $r$   $\triangleright$  no further call
9:   else
10:     $y \leftarrow \text{answer}(r, \sigma_U)$   $\triangleright$  full expensive call
11:    if  $y = \text{YES}$  then Ans  $\leftarrow$  Ans  $\cup \{r\}$ 
12:  end if
13: end if
14: end for
15: return Ans

```

3.7 CASTRUMMER: cascaded, structurally filtered batch join

CASTRUMMER makes three changes relative to the Trummer baseline, each targeting one of the cost or quality gaps identified in §3.4:

- (1) **Structural filter.** An explicit σ_S filter runs before any LLM call, exactly as in the SUQL baseline. This removes the incomplete-coverage failure: every unit the LLM ever sees already satisfies the structured condition, so no context budget inside a call is spent on rows that could have been rejected for free.
- (2) **Batched cheap pass with a three-way label.** Surviving rows are grouped into batches of $B=8$ and the LLM is asked for a direct YES/NO/UNCERTAIN label per row in one call. This targets the conflation failure directly: the model is no longer penalized for expressing ambiguity, so rows with indirect or diffuse semantic evidence are routed to escalation instead of being silently defaulted to NO.
- (3) **Batched expensive verification.** Rows labeled UNCERTAIN – as well as, symmetrically, rows whose cheap label log-odds fall inside $[\tau_-, \tau_+]$ under the same calibration machinery as SUQL v1 (§3.8) – are re-batched (again at $B=8$) into an expensive stage.

The combination is why CASTRUMMER improves cost *and* quality over its own baseline simultaneously, rather than trading one

for the other as CASUQL does: the structural filter shrinks the pool before any LLM call is issued at all, batching amortizes a shared prompt prefix across B rows per call on both the cheap and the expensive pass. So, unlike CASUQL, the cheap stage's wall-clock cost is a fraction of a full-context. And the verification pass repairs specifically the baseline's recall loss without re-processing rows the cheap pass was already confident about. The batch size $B=8$ is fixed across both passes in this version; we discuss adaptively sizing the verification batch as possible future work.

Algorithm 2 CASTRUMMER (batch-level cascade, $B=8$)

Require: rows R_σ surviving σ_S ; batch size B ; band $[\tau_-, \tau_+]$

```

1: Ans  $\leftarrow \emptyset$ ; Uncertain  $\leftarrow \emptyset$ 
2: for batch  $b \subset R_\sigma$  of size  $B$  do  $\triangleright$  one call per batch
3:    $\{(r, y_r, p_r)\}_{r \in b} \leftarrow \text{Cheap}(b, \sigma_U)$   $\triangleright$  joint
   YES/NO/UNCERTAIN + logprob per row
4:   for  $r \in b$  do
5:      $\ell_r \leftarrow \log \frac{p_r}{1-p_r}$ 
6:     if  $y_r = \text{UNCERTAIN}$  or  $\ell_r \in [\tau_-, \tau_+]$  then
7:       Uncertain  $\leftarrow$  Uncertain  $\cup \{r\}$ 
8:     else if  $y_r = \text{YES}$  then
9:       Ans  $\leftarrow$  Ans  $\cup \{r\}$ 
10:    end if
11:  end for
12: end for
13: for batch  $b' \subset$  Uncertain of size  $B$  do  $\triangleright$  expensive
   re-verification, same batch size
14:    $\{(r, y_r)\}_{r \in b'} \leftarrow \text{Expensive}(b', \sigma_U)$ 
15:   for  $r \in b'$  do
16:     if  $y_r = \text{YES}$  then Ans  $\leftarrow$  Ans  $\cup \{r\}$ 
17:   end if
18: end for
19: end for
20: return Ans

```

3.8 Calibrating the Cascade

Log-odds and the ambiguity band. The cheap stage returns $p_i \in (0, 1)$; we work in log-odds space, $\ell_i = \log \frac{p_i}{1-p_i}$, additive under independent evidence combination and symmetric around $\ell_i = 0$. We choose the band from a held-out calibration set C ($|C|=20$).

Beta-posterior bounds. For a candidate threshold τ , let $A_\tau = \{r \in C : \ell(r) \geq \tau\}$ be the calibration rows the cheap stage would accept at that threshold, and let $TP(\tau)$, $FN(\tau)$, $FP(\tau)$ count its true positives, false negatives, and false positives against the calibration set's ground truth.

We treat each calibration row's outcome as an independent Bernoulli trial and put a uniform (Beta(1, 1)) prior over the true recall r – i.e. before seeing any data, every value in $(0, 1)$ is equally plausible. After observing $TP(\tau)$ successes and $FN(\tau)$ failures, standard Beta-Binomial conjugacy updates this to the posterior

$$r \mid TP, FN \sim \text{Beta}(1+TP(\tau), 1+FN(\tau)).$$

This is a full distribution over what the true recall could be, and it narrows as the calibration set supplies more evidence for that particular τ . From it we take a *lower credible bound* $\widehat{r}_\alpha(\tau)$: the value

below which only a $(1-\alpha)$ share of the posterior’s probability mass falls, so that recall is at least $\hat{\tau}_\alpha(\tau)$ with posterior probability α (we use $\alpha=0.9$):

$$\hat{\tau}_\alpha(\tau) = F_{\text{Beta}(1+TP(\tau), 1+FN(\tau))}^{-1}(1-\alpha),$$

where $F_{\text{Beta}(a,b)}^{-1}$ is the inverse CDF (quantile function) of $\text{Beta}(a, b)$. This is exactly why “credible” is the right word and not a loose figure of speech: $\hat{\tau}_\alpha(\tau)$ is a standard Bayesian credible bound with a precise probabilistic meaning under the stated prior and Binomial model, not a hand-tuned safety margin. It is also self-correcting for how much evidence backs a given τ – a threshold supported by only two calibration rows (even if both are correct) gets pulled far below 1.0, while one backed by fifteen correct rows barely moves, which a bare point estimate would not distinguish.

The precision bound $\hat{p}_\alpha(\tau)$ is defined the same way from $\text{Beta}(1+TP(\tau), 1+FP(\tau))$. Given a target quality level Q^* (we use $Q^*=0.9$), the escalation band is the widest interval on either side of certainty that still clears this credible-bound test:

$$\tau_+ = \min\{\tau : \hat{p}_\alpha(\tau) \geq Q^*\}, \quad \tau_- = \max\{\tau : \hat{\tau}_\alpha(\tau) \geq Q^*\}.$$

A row with $\ell \geq \tau_+$ is accepted on the cheap score alone because precision is credibly high enough there; a row with $\ell \leq \tau_-$ is rejected because recall is credibly high enough for rejection to be safe; everything in between is escalated to the expensive stage precisely because neither bound clears Q^* yet.

Operator ordering. For k independent selection operators $O_1, \dots, O_k$, each with per-call cost c_i and selectivity $s_i \in (0, 1)$ (the probability a unit survives O_i and still has to pass the next filter), evaluated in the order $O_{\pi(1)}, \dots, O_{\pi(k)}$, the expected cost per unit is

$$\mathbb{E}[\text{cost}] = \sum_{j=1}^k c_{\pi(j)} \prod_{m < j} s_{\pi(m)}.$$

The term $\prod_{m < j} s_{\pi(m)}$ is the probability a unit survives every operator scheduled before $O_{\pi(j)}$ and therefore actually reaches it – so an operator placed late only pays its cost on the fraction of units still standing, while one placed early pays its cost on (nearly) everything. This sum is minimized by sorting the operators by ascending $c_i/(1-s_i)$: run the filters that are both cheap and good at rejecting first, since every unit one of them rejects saves the cost of every filter still waiting behind it – following the same rule ELEET [13] does. This is exactly why CAS_{TRUMMER}’s three filters are ordered the way they are: the structural filter runs first because it is free and typically rejects most of the pool, the batched cheap pass runs second, and the batched expensive verification runs last, since it only ever sees the small residual and is indeed the “heaviest.”

Complexity. Let $N = |R|$ be the dataset size, n the number of questions, and $s_S \in (0, 1)$ the per-question structured selectivity, so $|R_\sigma| = s_S N$. Total LLM calls:

- **Extraction + σ_S :** $O(n)$ calls – one per question; σ_S itself is a free SQLite scan.
- **Row-wise (SUQL, CAS_{SUQL}):** $O(n s_S N)$ calls – one per surviving row, per question.
- **Batched (Trummer, CAS_{TRUMMER}):** $O(n s_S N/B)$ calls – B rows packed per call.

- **Cascade overhead:** an extra $O(e \cdot n s_S N/B)$ expensive batch calls, where $e \ll 1$ is the fraction the cheap stage escalates.

All four terms are linear in n and N ; s_S , B , and e are the constants our design controls (via operator ordering, batching, and calibration, respectively).

Returning to the running example of §3.1: CAS_{SUQL} and CAS_{TRUMMER} close the same gap from different starting points. SUQL’s baseline already reaches the correct answer on that example, so CAS_{SUQL} does not change *what* it decides, only how much it costs to decide it – the cheap stage resolves tt0100 and tt0101 on a calibrated score alone whenever the band allows, escalating to a full answer() call only when confidence is genuinely ambiguous. Trummer’s baseline is precisely where the failure in Figure 1 comes from: tt0102 is admitted because σ_S was never applied before the join, and the model had no separate signal to tell a wrong-year row from a correct one. CAS_{TRUMMER} removes that failure at its source rather than compensating for it after the fact: the structural filter it adds runs before any LLM call, so tt0102 is rejected for free exactly as it is under SUQL, and the batched cheap pass’s three-way YES/NO/UNCERTAIN label gives genuinely ambiguous rows a way to ask for a second opinion instead of being silently guessed. Both cascades sit on top of a baseline that already handles $\sigma_S \wedge \sigma_U$ correctly for CAS_{SUQL}; CAS_{TRUMMER} has to earn that correctness rather than inherit it, which is why §4.2 shows it improving cost *and* quality at once, while CAS_{SUQL} only ever had cost left to improve.

4 EXPERIMENTS

4.1 Experimental Setup

Datasets. *IMDb* (primary, §4.2): ten held-out, genre-diverse questions (10q suite), each with its own 100-movie pool drawn from a $\sim 25,000$ -row IMDb corpus: 12 true positives, 28 structured-semantic negatives, 30 semantic-only distractors, 30 double negatives; 40 rows satisfy the structured predicate in every question. *Amazon Fashion* (cross-domain validation, §4.3): the 2018 McAuley Lab release ($\sim 186,000$ products, $\sim 882,000$ reviews), same questions suite design. **Models.** Ollama-served open-weight models (cheap: Gemma 4 e2b, expensive: Gemma 4 26b) used for both datasets. **Repetitions.** Each of the four methods is run 10× per question with further averaging. **Metrics.** Precision, recall, F1 against hand-picked ground truth rows; LLM calls/time(sec) with cheap/expensive split. **Questions.** Each question decomposes into one or two structured filters (a genre tag, a year or runtime range) and one semantic condition stated in natural language (e.g. “reviews praising the characters’ chemistry,” “reviews calling the film thought-provoking”); the ten IMDb questions span seven genres and range from direct sentiment cues (*funny*, *hilarious*) to more abstract, paraphrase-prone ones (*chemistry*, *approval of a twist*).

4.2 Results

Cascading the row-wise strategy (CAS_{SUQL}) trades time for a modest quality gain: 7.99× slower wall time for +2.7% F1 and +6.6% recall. The batched join (Trummer) baseline is far cheaper but at a real quality cost (F1 0.444, worst of the four). Cascading the batch join strategy (CAS_{TRUMMER}) improves both cost and quality over

Metric	SUQL base.	CasSUQL	Trummer base.	CasTRUMMER
Precision	0.683 $\pm$ 0.159	0.709 $\pm$ 0.199	0.655 $\pm$ 0.190	0.810 $\pm$ 0.218
Recall	0.758 $\pm$ 0.256	0.808 $\pm$ 0.261	0.408 $\pm$ 0.227	0.750 $\pm$ 0.180
F1	0.678 $\pm$ 0.171	0.696 $\pm$ 0.174	0.444 $\pm$ 0.153	0.757 $\pm$ 0.159
Wall time (s)	115.57 $\pm$ 12.28	923.60 $\pm$ 250.40	71.28 $\pm$ 48.26	35.05 $\pm$ 1.86
LLM calls	117.08 $\pm$ 7.48	70.05 $\pm$ 12.28	96.87 $\pm$ 78.32	10.00 $\pm$ 0.00

Table 1: Cost and quality per question, mean $\pm$ std. dev. across the 10 questions (10 reps each).

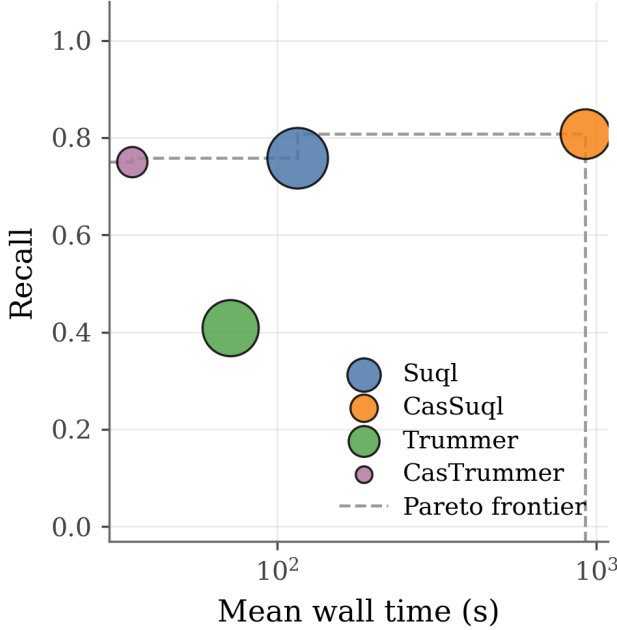


Figure 4: IMDb 10q: recall vs. mean wall time (log scale); marker area is proportional to mean LLM calls. The dashed Pareto frontier connects the three non-dominated methods – CasTRUMMER, SUQL baseline, and CasSUQL, in increasing order of time – while Trummer baseline sits strictly below it, beaten on both axes at once.

its own baseline: 2.03 $\times$ faster, 10 $\times$ fewer calls, +70% F1 and, relative to CasSUQL, is 26.3 $\times$ faster and uses 1/7 the calls while still gaining +8.8% F1, at a $\approx$ 7% relative recall cost. The results can be observed on Table 1. Overall, for IMDb dataset CasTRUMMER had the best cost-quality balance: in the recall-vs-time plot the best approach is located in the upper-left corner, and CasTRUMMER anchors the low-time end of the Pareto frontier there (Figure 4).

Robustness across questions. CasTRUMMER is the only method that doesn’t collapse on the hardest question in the suite,

“Which Romance movies between 90 and 130 minutes have reviews praising the characters’ chemistry or relationship?”

where it reaches F1 0.917 while every other method collapses (SUQL baseline 0.267, CasSUQL 0.286, Trummer baseline 0.154).

	IMDb 10q	Amazon Fashion 10q
Δ F1	+11.7%	−4.3%
Δ calls	−91.5%	−73.6%
Δ time	−69.7%	−39.8%

Table 2: The best-performing cascade(CasTRUMMER) against the best-performing baseline(SUQL baseline) overall (full per-method numbers in Table 3).

Metric	SUQL base.	CasSUQL	Trummer base.	CasTRUMMER
Precision	0.789 $\pm$ 0.181	0.764 $\pm$ 0.196	0.330 $\pm$ 0.061	0.753 $\pm$ 0.167
Recall	0.927 $\pm$ 0.102	0.833 $\pm$ 0.189	0.967 $\pm$ 0.047	0.878 $\pm$ 0.114
F1	0.835 $\pm$ 0.112	0.776 $\pm$ 0.149	0.488 $\pm$ 0.068	0.799 $\pm$ 0.130
Wall time (s)	19.66 $\pm$ 1.96	48.72 $\pm$ 6.37	36.82 $\pm$ 18.03	11.82 $\pm$ 1.75
LLM calls	49.20 $\pm$ 1.03	78.30 $\pm$ 14.55	38.01 $\pm$ 19.17	13.00 $\pm$ 2.15
\$-cost	0.0164 $\pm$ 0.0016	0.0406 $\pm$ 0.0053	0.0306 $\pm$ 0.0150	0.0098 $\pm$ 0.0015

Table 3: Amazon Fashion 10q, Gemma: cost and quality per question, mean $\pm$ std. dev. across the 10 questions (10 reps each).

4.3 Cross-Domain and Cross-Model Validation

To test whether the IMDb findings above generalize to other domains, we repeat the same 10q experiment on Amazon Fashion reviews.

Table 2 illustrates that the cost gain is preserved over the domains, while quality doesn’t: CasTRUMMER loses 4.3% on F1. But it is a small price to pay for the preserved cost gain: $\sim$ 74% on calls and $\sim$ 40% on time. The full results of the experiment over Amazon fashion may be observed in Table 3.

4.4 Cost at Scale: From Ten Queries to Hundreds

§4.2 and §4.3 average over a 10-question suite. A natural question for anyone deploying either baseline or either cascade is how cost would change if the workload were 100 or 1,000 questions instead of 10, and whether the fixed 100-row candidate pool used throughout this paper is itself a safe size to generalize from. We answer both using data already collected in this project rather than fresh assumptions: a linear extrapolation of Table 1’s per-question means, and an existing pool-size sweep run on Amazon Fashion that varies a second, orthogonal scale parameter.

Extrapolating over the number of queries. Every query in our benchmark is answered independently end to end: there is no result caching, no shared prompt prefix, and no batching of multiple questions into one LLM call (batching, where it happens at all, is only ever across the rows of a single question). Consequently, both the LLM call count and the compute-time-based dollar cost scale exactly linearly in the number of queries N : $\text{Calls}(N) = N \cdot \text{calls}$ and $\text{Cost}(N) = N \cdot \text{cost}$, using the per-question means already reported in Table 1. Wall-clock time is the one axis where this is a worst case rather than a hard limit: because each query’s evaluation is independent of every other’s, a deployment free to run W of them concurrently sees $\text{Time}(N) \approx N \cdot t/W$, whereas calls and dollar cost are incurred once per query and do not shrink under concurrency the way wall-clock does.

Method	Sequential wall time		LLM calls	
	$N=100$	$N=1,000$	$N=100$	$N=1,000$
SUQL base.	3.2 h	32.1 h	11,708	117,080
CASUQL	25.7 h	256.6 h	7,005	70,050
Trummer base.	2.0 h	19.8 h	9,687	96,870
CASTRUMMER	1.0 h	9.7 h	1,000	10,000

Table 4: Linear extrapolation of Table 1’s per-question means to N queries, run sequentially (no cross-query batching, caching, or parallelism assumed).

At $N=1,000$ sequential queries, the sequential-time gap between the cheapest and most expensive method compounds from a difference invisible at $N=10$ into one exceeding ten days of wall-clock time (CASUQL) versus under half a day (CASTRUMMER) – the point at which the ranking stops being a matter of a few minutes’ patience and starts deciding whether a workload is interactive or an overnight batch job. Because calls and dollar cost are strictly linear regardless of concurrency, CASTRUMMER’s call-count advantage over both baselines, and over CASUQL, holds at exactly the same ratio at any N , including $N=1$: linear extrapolation changes the stakes of §4.2’s ranking, not the ranking itself.

The pool-size axis is the one that actually changes the ranking. The projection above holds the per-query candidate pool fixed at the 100 rows used throughout §4. A separate scaling study already run on Amazon Fashion (one question, the same Gemma model pair, candidate-pool size $n \in \{25, 50, 100, 200, 400, 800\}$) probes the axis a query-count extrapolation cannot: what happens as the pool a *single* query has to search grows, independent of how many queries are asked. Two findings from that sweep matter for generalizing beyond this paper’s $n=100$ setting. First, CASTRUMMER’s call- and time-savings over the SUQL baseline hold at every n tested, from 25 to 800 – so the cost advantage reported in Table 1 is not an artifact of this paper’s particular pool size. Second, its quality advantage is pool-size dependent: F1 is 0.00 at $n=25$ and 0.40 at $n=50$ (the fixed 20-row calibration budget is too large relative to a tiny pool for the threshold search in §3.8 to produce a usable band), climbing to 0.77 at $n=100$ and matching or exceeding the baseline (0.89–0.92) only from $n \approx 200$ onward. CASTRUMMER becomes *cheaper* than the baseline at $n \approx 25$, but only becomes cheaper *and* reliably close to baseline quality at $n \gtrsim 100$ – which is exactly the pool size this paper’s main benchmark happens to use, not a coincidence-free safety margin.

Practical reading. A deployment that scales query *volume* while keeping each query’s candidate pool comparable to this paper’s ($n \approx 100$ or larger) can treat §4.2’s ranking as safe to extrapolate: CASTRUMMER’s call and dollar-cost advantages are architectural and grow in absolute terms with N , and its wall-clock advantage compounds further under any degree of query-level parallelism. A deployment that instead scales the size of the corpus each individual query searches should treat $n \approx 100$ as a lower bound on where the calibrated cascade’s advantage has actually been observed, not an incidental detail of our experimental setup.

5 CONCLUSION AND FUTURE WORK

This paper reconciled two execution strategies for a semantic predicate over structured data that trade off in opposite directions – an

accurate but call-hungry row-wise filter and a cheap but attention-starved batched join – by wrapping each in a cascade whose acceptance band comes from a Beta-posterior credible bound rather than a hand-tuned cutoff. Cascading the batched join is where this pays off most cleanly, since its shared prompt prefix makes a second opinion on an ambiguous case nearly free; cascading the row-wise filter does not enjoy the same benefit, since it has no batch to amortize that second opinion across. The cross-domain check further showed that this architecture’s efficiency gain travels regardless of domain while its quality gain depends on the underlying text carrying enough signal for a cheap pass to separate confident cases from ambiguous ones. Several of the choices we made to keep this study tractable are, correspondingly, the most direct next steps.

Making the cheap stage viable on short-form text. The Amazon Fashion result (§4.3) suggests the calibrated band degrades gracefully rather than failing outright when reviews are short, but does not recover the accuracy advantage seen on longer text. A natural next step is to change what the cheap stage is shown rather than the calibration procedure itself – e.g. aggregating several short reviews per candidate into one synthetic passage before scoring it, or letting the calibration set adapt its own size to how much signal a domain’s text actually carries, rather than reusing the same fixed budget across domains of very different verbosity.

Joining the two scale axes we studied separately. §4.4 treats query-count scaling and candidate-pool-size scaling as independent, because the evidence we had for each came from different experiments – a linear extrapolation for the former, a single-question sweep on one domain for the latter. A joint study (many questions, several pool sizes, both domains) would test whether the pool size at which CASTRUMMER’s quality stabilizes is a property of the method or an artifact of the question we happened to sweep it on. The concurrency assumption behind our query-count projection is similarly untested: we assumed independent queries divide wall-clock time by the number of workers, but never measured whether a single Ollama-served backend actually delivers that under real concurrent load.

Completing the deferred KV-cache-aware stage. Stretto’s execution ladder (§2) remains outside this paper’s scope because it needs direct attention-state access our Ollama-served backend does not expose. Migrating to a backend such as vllm would let us test whether that ladder composes with the Beta-posterior band studied here as a third, finer-grained level of the cascade, rather than a replacement for it.

Broadening what we vary. This paper fixes one model pair and studies two domains; extending the same benchmark to other model families and sizes, and sweeping structured selectivity directly rather than only candidate-pool size, would test whether the operator-ordering rule of §3.8 keeps recommending the same stage order once the assumptions behind it – fixed selectivity, one specific pair of models – are allowed to move.

Pricing against billed inference. The dollar-cost figures throughout this paper are a compute-time proxy for locally served, open-weight models, not a token-metered price. Replaying the same benchmark against a hosted, per-token-billed API would show whether the cost ranking in §4.2 reflects something about the methods themselves or about how we chose to price local compute time.

REFERENCES

- [1] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E. Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2SQL is Not Enough: Unifying AI and Databases with TAG. *arXiv preprint arXiv:2408.14717* (2024).
- [2] Surajit Chaudhuri and Kyuseok Shim. 1996. Optimization of Queries with User-Defined Predicates. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*. 87–98.
- [3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *arXiv preprint arXiv:2308.15363* (2023).
- [4] Joseph M. Hellerstein. 1998. Optimization Techniques for Queries with Expensive Methods. *ACM Transactions on Database Systems* 23, 2 (1998), 113–157.
- [5] Joseph M. Hellerstein and Michael Stonebraker. 1993. Predicate Migration: Optimizing Queries with Expensive Predicates. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. 267–276.
- [6] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A Declarative System for Optimizing AI Workloads. *arXiv preprint arXiv:2405.14696* (2024). System later presented as Palimpsest at CIDR 2025.
- [7] Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina J. Semnani, Chen Jie Yu, and Monica S. Lam. 2023. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. *arXiv preprint arXiv:2311.09818* (2023). Also in Findings of the Association for Computational Linguistics: NAACL 2024.
- [8] Priti Mishra and Margaret H. Eich. 1992. Join Processing in Relational Databases. *Comput. Surveys* 24, 1 (1992), 63–113.
- [9] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. LOTUS: Enabling Semantic Queries with LLMs Over Tables of Unstructured and Structured Data. *arXiv preprint arXiv:2407.11418* (2024).
- [10] Gabriele Sanmartino, Matthias Urban, Paolo Papotti, and Carsten Binnig. 2026. The Stretto Execution Engine for LLM-Augmented Data Systems. *arXiv preprint arXiv:2602.04430* (2026).
- [11] P. Griffiths Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. 23–34.
- [12] Immanuel Trummer. 2025. Implementing Semantic Join Operators Efficiently. *arXiv preprint arXiv:2510.08489* (2025).
- [13] Matthias Urban and Carsten Binnig. 2024. ELEET: Efficient Learned Query Execution over Text and Tables. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4867–4880.